

dr. Matej Šprogar

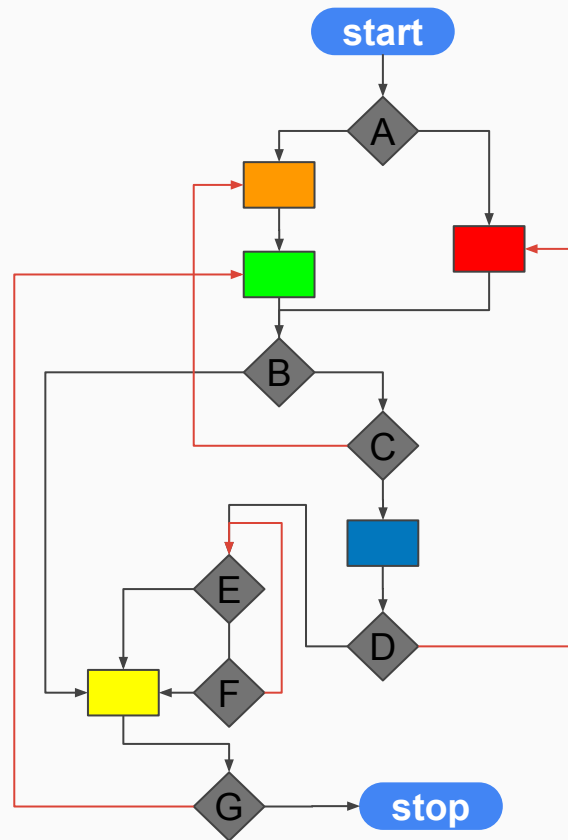
Strukturirano programiranje

Poglavje o pisanju razumljivih programov



Problem

če pogoj A potem
 oranžno
 zeleno
 sicer
 rdečo
 konec A
 če pogoj B potem
 rumeno
 če pogoj G potem
 skoči na zeleno
 sicer
 stop
 konec B
 sicer
 če pogoj C potem
 skoči na oranžno
 sicer
 modro
 če pogoj D potem
 skoči na rdečo
 sicer
 če pogoj E potem
 skoči na rumeno
 sicer
 če pogoj F potem
 skoči na rumeno
 sicer
 skoči na test E
 konec F
 konec C
 konec D
 konec E
 konec F



Raje programski skoki ali programska struktura?

Zapis algoritma z diagrami poteka je težaven, ker puščice prelahko omogočajo preskoke med *konteksti* in s tem zmedejo programerja. Isti problem ima tudi strojni in vsi programski jeziki ter psevdokod, ki dovoljujejo skoke med programskimi ukazi.

Strukturirano programiranje govori o zapisu algoritma s pomočjo temeljnih *programskih struktur* in brez vidnega *goto* stavka (o *podatkovnih strukturah* boste govorili drugje).

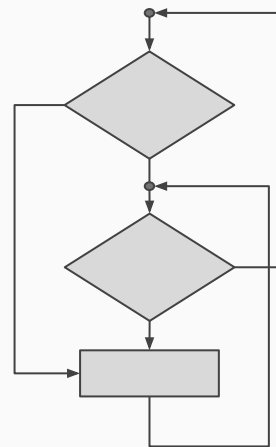
Strukturirani algoritmi so preglednejši, razumljivejši in zato varnejši. Ukaz *goto* vsaj teoretično ni več potreben, čeprav je v določenih primerih elegantnejša rešitev in zato ostaja v C++ in C# tudi v čisti obliki, Java pa ga ima zgolj v zamaskirani obliki.

Strukturirano programiranje

Strukturirano programiranje zagovarja **sekvenčno** uporabo programskih struktur:

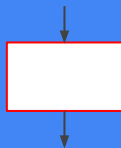
1. **stavek** (celovit kos kode) in
2. **selekcija** (izbira poti skozi program),
3. **iteracija** (ponavljanje),
4. **podprogram** (ponovno uporaben in prilagodljiv blok kode).

Zaporedje teh struktur lahko opiše katerikoli algoritem, predvsem pa ga je **lažje*** brati kot alternativne zapise algoritmov.



Vseh diagramov poteka sicer ne moremo enostavno zapisati s programskimi strukturami.

Stavčni blok



Enega ali več stavkov lahko smatramo kot **blok**, ki ima natanko **eno** vhodno in **eno** izhodno točko.

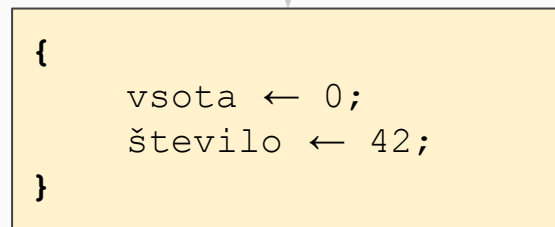
Bloki si sledijo v zaporedju, kot ga določajo puščice, stavki v bloku pa od zgoraj navzdol, kot odstavki v besedilu.

V stavčnem bloku so lahko *vgnezdeni* tudi drugi (pod)bloki / stavki.



Stavke bloka v C++, Javi in C# obdamo z zavirami oklepaji { };

Python nasprotno **zahteva** enakomeren *zamik*. (Tudi sicer so zamiki nujni/koristni za berljivost kode!)



Stavek

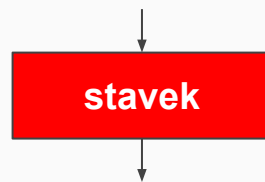
Vse temelji na **stavku** - ukazu procesorju. Stavek tipično spremeni pomnilnik in vpliva na program. Stavek brez posledic je **nekoristen**.

Stavki so lahko ELEMENTARNI:

1. prireditve (pisanje)
2. uporaba (branje)

ali SESTAVLJENI (stavčni blok):

1. zaporedje (sekvenca)
2. odločitev (izbira, selekcija)
3. ponavljanje (iteracija)
4. gnezdenje



stavek

```
SVPG ← 40 + 2
```

```
ODGOVOR ← SVPG
```

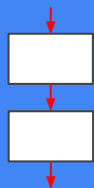
```
izpiši ODGOVOR
```

```
sortiraj
```

```
ustvari datoteko
```

```
...
```

Sekvenca



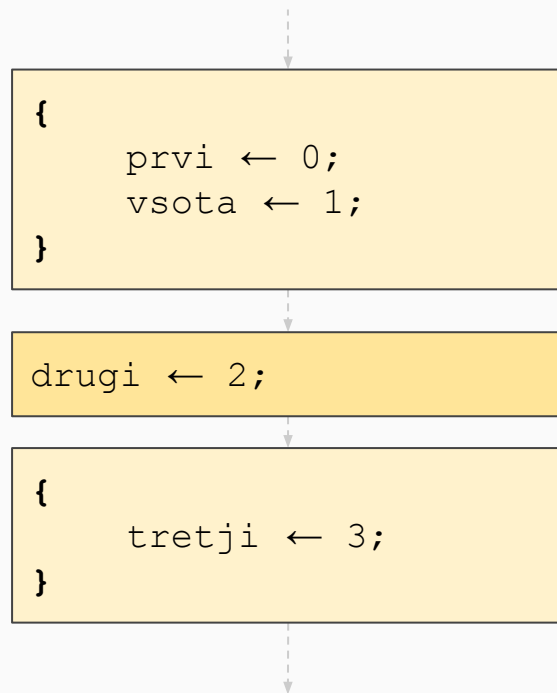
... je vsako zaporedno izvajanje ukaznih stavkov ali blokov.

Izvajanje elementov ena po ena tvori **izvajalno pot** programa, sekvenca pa je *najenostavnejša* izvajalna pot.

Sekvenca vedno izvede vse ukaze v istem vrstnem redu.

(Pre)Dolgim sekvencam pravimo tudi "špageti" koda, ki ni zaželena, saj nakazuje nepotrebno ponavljanje ukazov, jo je težko brati, vzdrževati in popravljati.

V sekvenci so lahko posamezni stavki ali drugi bloki; skoznjo gre natanko ena *pot*!



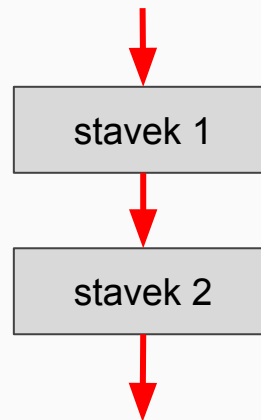
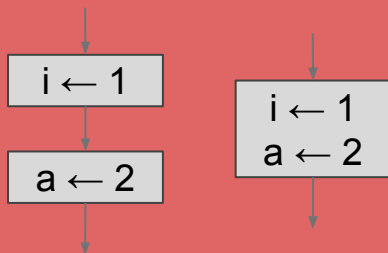
Sekvenca

Povezavo aktivnosti s puščicami lahko ponazorimo z zaporedjem ukazov v točnem vrstem redu - s *sekvenco*.

Odpri pivo.

Spij pivo.

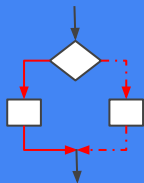
Zaporedje elementarnih stavkov istega tipa lahko poenostavimo v skupni blok in tako skrajšamo zapis:



stavek 1
stavek 2

Opišimo postopek za pozdravljanje v odvisnosti od ure dneva.

Odločitev



Navadno izvajanje zaporednih korakov bo šlo *vedno* po isti poti: od prvega do zadnjega koraka, ne glede na karkoli. Ker to ni dovolj, je edina rešitev skok na "nenaslednji" (tretji) ukaz!

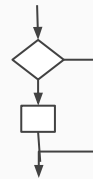
Odločitev je najenostavnejši konstrukt, ki omogoča skok na določeno točko programa (prevajalnik jo izvede kot goto/skok strojni ukaz (*jmp*)).

Odločitev je minimalni skok naprej v programski kodi.

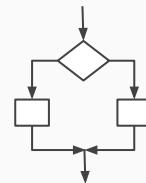
Pri večini problemov nam enostavno zaporedje ukazov ne zadostuje, saj moramo občasno izbrati (selekcija) eno izmed alternativnih poti izvajanja programa.

V programskih jeziki izbiro izvedemo z odločitvenim stavkom.

Če še ni poldne, pojem zajtrk.



Če je lepo vreme grem na tekmo, sicer ostanem doma.

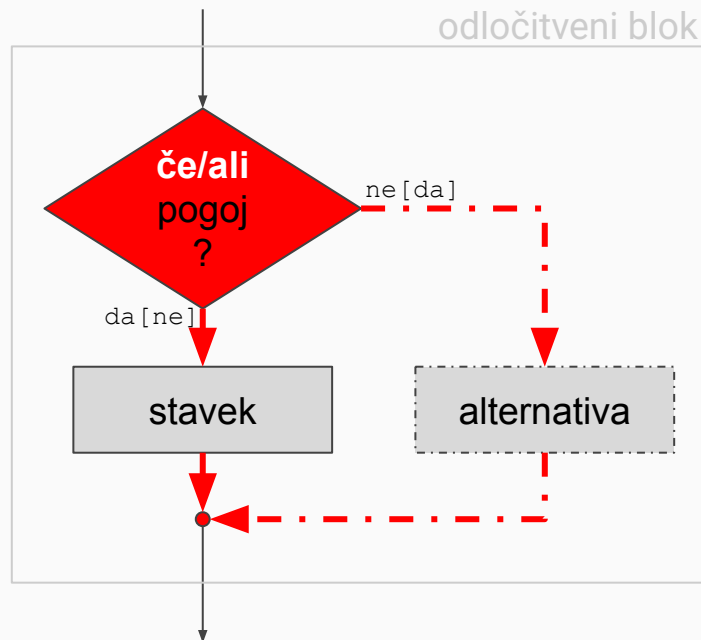


Odločitev

Pogosto enostavno zaporedje navodil ni dovolj. Potrebno je izbrati pravo pot skozi program. Vejitev v diagramu poteka v psevdokodu nadomesti odločitveni stavek.

V primeru odločitve se izvede **ali** stavek **ali** (če obstaja) alternativa, nikoli pa oba!

Pogoj naj bo razumljiv (če *vrema je lepo* ...)! Negacija v pogojih je težje razumljiva in v toliko slabša (če *ni vreme je grdo potem* ...)



če je[ni] pogoj potem
stavek
[sicer
alternativa]
konec odločitve

If stavek

Odločitev temelji na pogoju - logičnem izrazu z vrednostjo *true* ali *false*.

Programski konstrukt je tim. **if** stavek, ki lahko zavzame dve obliki. Prva, krajša, izvede *blok_true* samo v primeru, da je pogoj resničen, sicer ga preskoči.

V drugi obliki (*if-else*) se izvede natanko **ena** od dveh alternativnih poti.

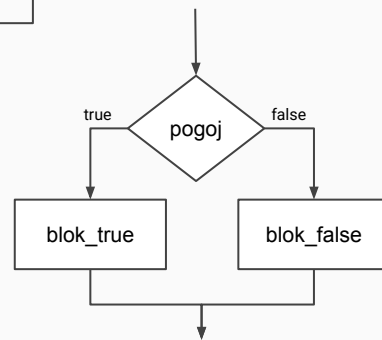
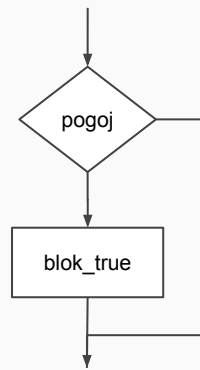
If nadzira le **prvega** od sledečih mu stavkov; če jih mora več, jih moramo z zavitimi oklepaji { } združiti v blok.

```
if (pogoj)
    stavek_true;
```

```
if (pogoj)
    stavek_true;
else
    stavek_false;
```

```
if (ura < 12)
    jutro ← true;
```

```
if (ura > 18) {
    jutro ← false;
    večer ← true;
}
```



Veriženje in gnezdenje odločitev

Odločitveni stavek lahko izbira tudi med več kot le dvema alternativama, če le pametno in previdno kombinirano **else** blok z novo odločitvijo...

Možno je *veriženje* izključujočih se odločitev ali *gnezdenje* prekrivajočih še odločitev ali (težje razumljiva) kombinacija obeh tehnik.

Bolj zapletene pogoje *moramo* lepo podpisovati in ograjevati z oklepaji.

Veriženje lahko nadomestimo tudi s posebnim **switch*** ukazom.

```
if (pogoj1) {                // primer veriženja
    blok_true1;
} else if (pogoj2) {
    blok_false1_true2;
} else {
    blok_false1_false2;
}
```

```
if (pogoj1) {                // primer gnezdenja
    blok_true1;
    if (pogoj2)
        blok_true1_true2;
    else
        blok_true1_false2;
}
else
    blok_false1;
```

Pogosta napaka

Pri uporabi gnezdenih in verižnih pogojev pogosto pride do napak zaradi napačnega podpisovanja, ker človek **vidi** kodo *drugače*, kot jo bere prevajalnik.

Čeprav zamiki nakazujejo drugače, se *else* nanaša na **zadnji** *if* stavek!

Takšnim težavam se izognemo z uporabo izrecnih blokov `{}`.

Namesto

```
if (a>=0)
    if (a>5)
        a_vecji_od_5_blok;
else
    a_manjsi_od_0_blok; // ?
```

bi morali napisati:

```
if (a>=0) {
    if (a>5)
        a_vecji_od_5_blok;
}
else
    a_manjsi_od_0_blok;
```

Gnezdenje ↔ Veriženje

Veriženje lahko enostavno zapišemo kot gnezdenje:

če (x > 0) pozitivno; sicer če (x = 0) nič; sicer negativno;	če (x > 0) pozitivno; sicer če (x = 0) nič; sicer negativno;
---	--

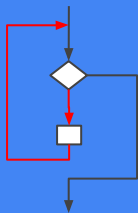
Pretvorba gnezdenja v veriženje je zahtevnejša:

če (x > 0) če (x sodo) pozitivno_sodo; sicer pozitivno_liho; sicer 0_ali_negativno;	če (x > 0 in x sodo) pozitivno_sodo; sicer če (x > 0 in x liho) pozitivno_liho; sicer 0_ali_negativno;
---	---

Kaj je "narobe" s spodnjim algoritmom?

```
izpisi 1  
izpisi 2  
izpisi 3  
.  
.  
.  
izpisi 42
```


Iteracija



Pogosto moramo isto sekvenco ukazov večkrat ponoviti. Število ponovitev je lahko znano vnaprej oziroma je odvisno od nekega pogoja.

Kakršnokoli ponavljanje (tako kode kot podatkov) je pri programiranju zelo **nezaželeno** in se mu *moramo* izogniti. Rešimo ga z **minimalnim** skokom nazaj.

Ponavljjanje kode zato spada v posebni konstrukt - **zanko**. Obstajajo tri klasične izvedbe zank:

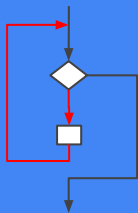
1. *while* zanka,
2. *for* zanka, in
3. *do-while* zanka

Zanka je v bistvu **pogojen** skok nazaj po programu; pogoj skoka se lahko preverja (1) **pred** telesom zanke (*for* in *while*), ali (2) **za** telesom zanke (*do-while*).

Telo zanke je tisti najmanjši delček kode, s katerim zanka zgradi celotno sekvenco.

Iteracija je nujna, ko sekvenca ni znane dolžine, in smiselna, če je zapis z zanko krajši kot brez nje.

While zanka



While je najbolj splošna zanka, s katero lahko opravimo **vse** iteracijske naloge.

Pogoj zanke je kakršenkoli logičen izraz; če je pogoj logična konstanta, se zanka ali sploh ne bo izvedla (*false*) ali pa bo neskončna zanka (*true*).

Obstajajo še posebni ukazi, ki ob uporabi znotraj telesa prekinajo zanko s tem, da *nadzorovano* skočijo iz zanke (*break*, *return*, *throw*). Več o njih drugje.

```
while (pogoj) {  
    telo_zanke;  
}
```

```
while (true) {  
    neskončno_ponavljan_blok_kode;  
}
```

```
while (false) {  
    nikoli_izveden_blok_kode;  
}
```

While zanka

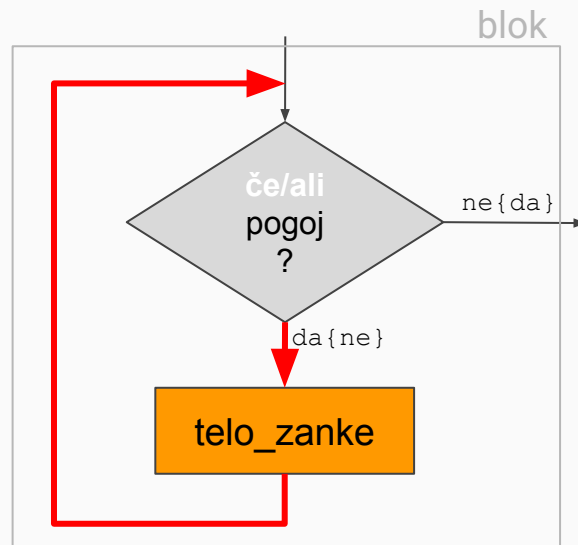
je najbolj splošna z univerzalnim pogojem.

POMEMBNO:

Zanka NAJPREJ preveri pogoj P za izvedbo telesa zanke, ki se zato lahko izvrši ničkrat, enkrat ali tudi večkrat!

while (P)
T;

Dokler je v steklenici še kaj ***ponavlja***
natoči nov kozarec
konec zanke



dokler pogoj **ponavlja**
telo_zanke
konec zanke

For zanka

Ker se v zankah pogosto potrebuje še kakšna pomožna spremenljivka, je *for* zanka **okrajšava** za daljši *while*.

Pri *for* lahko v isti vrstici dodatno definiramo še dva posebna stavka: *pripravljalnega* in *prehodnega*.

Prprava se izvede natančno **enkrat** pred vstopom v zanko, prehodni blok pa **po vsaki** izvedbi telesa zanke.

Spremenljivke, ki jih deklariramo v pripravljalnem bloku, so vidne in s tem uporabne le **znotraj** telesa zanke...

```
priprava;
while (pogoj) {
    telo_zanke;
    prehod;
}

for (priprava; pogoj; prehod) {
    telo_zanke;
}

for (int i=0; i<10; i=i+1) {
    samo_ta_blok_lahko_uporabi_i;
}
```

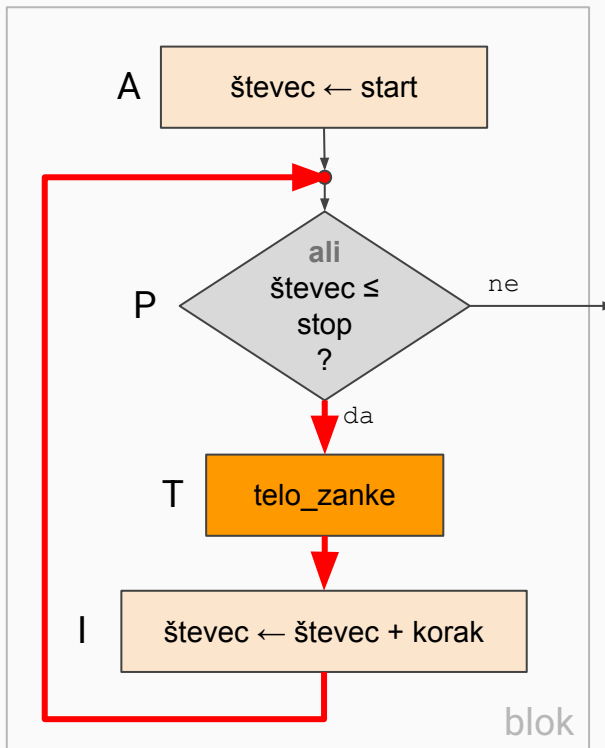
For zanka

Pogosto je v zanki pomembno vedeti, **kolikokrat** se je zanka že izvedla. Zanka **for** zato predvideva poseben pripravljalni blok A, v katerem lahko ustvarimo števec, s katerim nadziramo število izvedb (iteracij) telesa zanke T glede na pogoj P, in dodaten prehodni blok I za spremembo števca.

For zanka je enakovredna while zanki.

for (A; P; I)
T;

Primer: start = 3, stop = 7, korak = 1,
števec = {3, 4, 5, 6, 7, 8}



ponavljaj za števec od start do stop
s korakom korak
telo_zanke
konec zanke

Do-while

Zanka je uporabna za primere, ko se mora telo zanke zagotovo izvršiti **vsaj enkrat**.

Do-while ne more nadomestiti zanke while, obratno je seveda mogoče!

```
do {  
    telo_zanke;  
} while (pogoj);
```

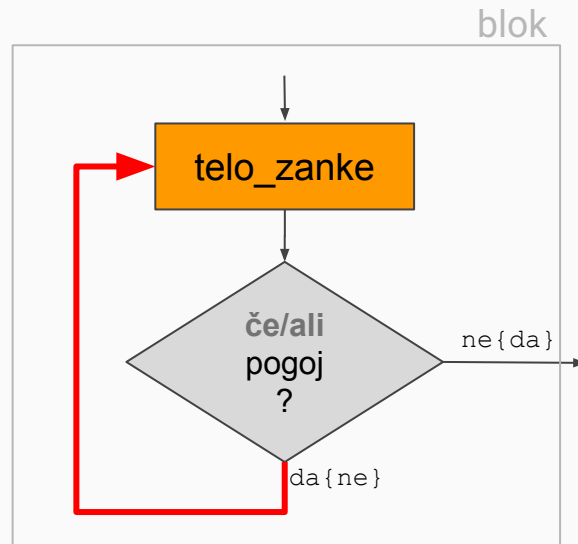
```
prvič ← true;  
while (prvič or pogoj) {  
    telo_zanke;  
    prvič ← false;  
}
```

Do-while zanka

je tretji tip zanke, ki se za razliko od *for* in *while* zank vedno izvrši **vsaj enkrat!**

Z *do-while* zanko NI MOGOČE implementirati drugih dveh zank, medtem ko lahko z *while* in *for* dosežemo tudi *do-while* učinek!

```
do {  
    T;  
} while (P);
```



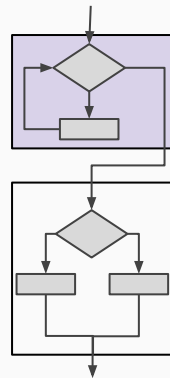
```
ponavljaaj  
    telo_zanke  
dokler pogoj
```

Sestavljanje elementov

Blok, kot smo ga uporabljali do sedaj, lahko predstavlja katerokoli enostavni stavek ali sestavljen blok ukazov.

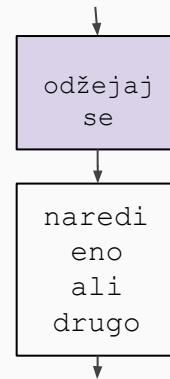
Odločitve in zanke so dejansko bloki kode, ki jih lahko sestavljamo v poljubna zaporedja, gnezdimo...

Težimo k *višjenivojskemu* opisu, saj je razumljivejši in tudi bolj berljiv.



```
dokler sem žejen ponavljaj  
    spiј kozarec vode  
konec zanke
```

```
če pogoj potem  
    delaj_prvo  
sicer  
    delaj_drugo  
konec odločitve
```



```
odžejaj se
```

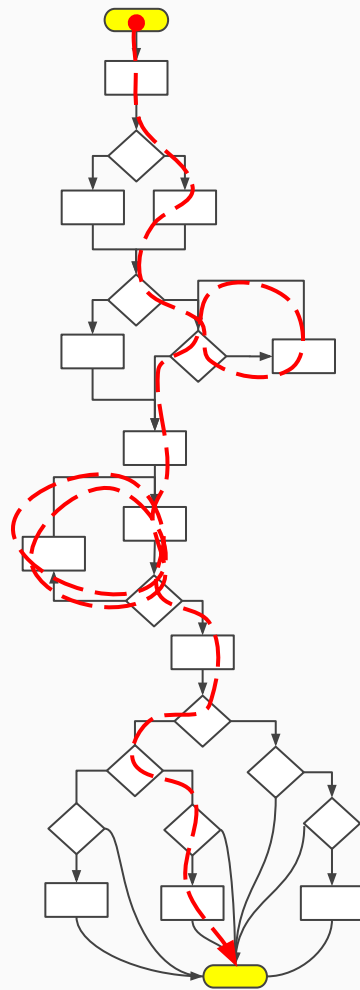
```
naredi eno ali  
    drugo
```


Izvajalna pot

Kombiniranje struktur omogoča opis še tako kompleksnih algoritmov in ukazuje procesorju izvajanje različnih sekvenc ukazov.

Vsaka od možnih sekvenc ukazov predstavlja neko možno **izvajalno pot** skozi program, ki ji bo procesor sledil. Katero pot bo izbral, je odvisno od **zunanjih** dejavnikov (vnešenih podatkov, informacij).

Na sliki je označen primer **ENE** izvajalne poti. Koliko *različnih* poti obstaja?



Povzetek:

Zakaj torej strukturirano programiranje?

Alternativa programskim strukturam je *goto* programiranje, ki je večinoma izredno nevarno, ampak nujno v nestrukturiranih jezikih (zbirniki, Basic, Fortran...).

Programske strukture lahko uporabljamo v različnih programskih paradigmah, kot sta *objektno* in *funkcijsko* programiranje. Dejansko jim ne nasprotujejo, ampak jih celo omogočajo.

Izkušnje nas učijo, da je najboljši tisti pristop, ki pobere najboljše iz vseh svetov in se ne odloči apriori za eno paradigmo, ampak jih omogoča več. Pametna* uporaba programskih struktur programu kvečjemu koristi.